C-PROGRAMMING (CSA0202)

CO4 ASSIGNMENT 4

Structures, Unions and Storage Classes

NAME: LEELAVATHI R

REG.NO: 1925260191

TASK 1 – DATA DESIGN REVIEW

1.1 Diagnose the Design

The original program uses parallel arrays to store patient details. The following are its major design deficiencies:

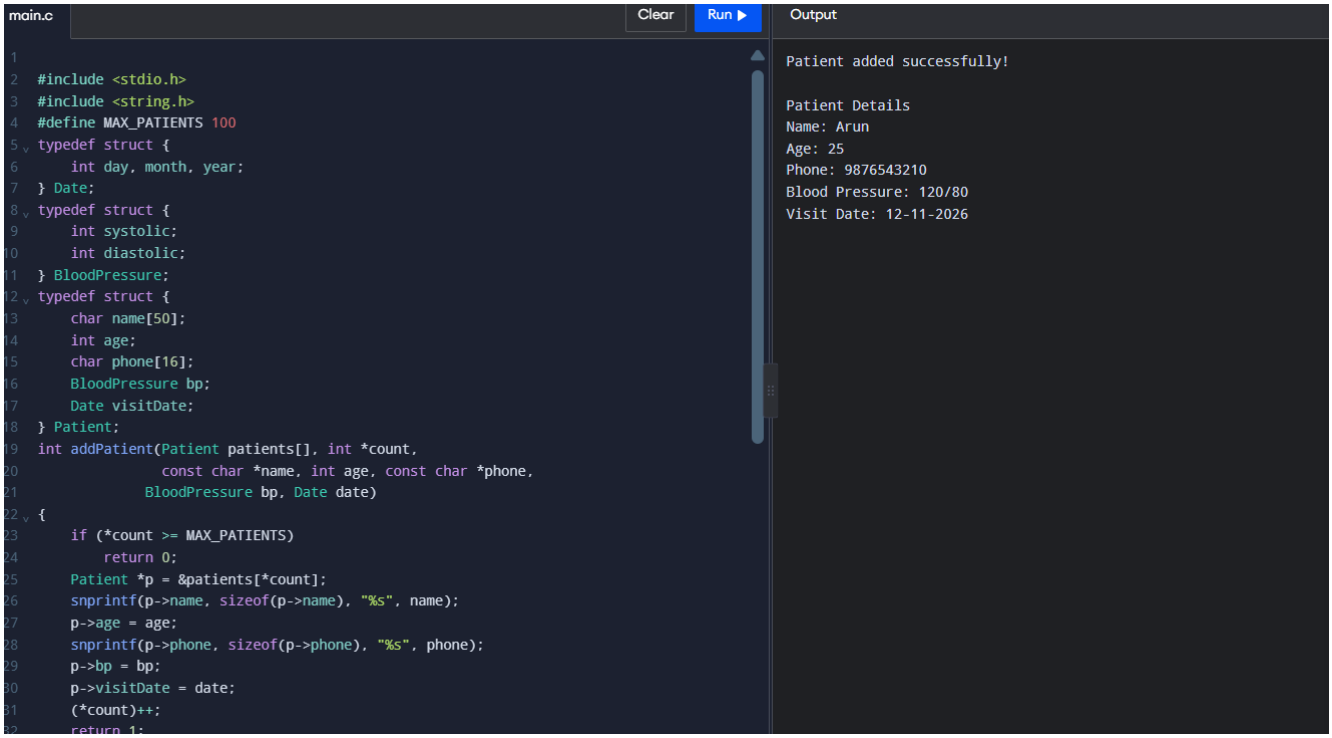
- Parallel arrays: Patient details are stored at the same index in different arrays. If one array is sorted or updated separately, the details may become mismatched. A patient could receive another patient's phone number or blood-pressure record.
- Phone number capacity: char phone[100][10] provides only 10 bytes per phone number. A 10-digit number also needs one extra byte for the terminating '\0'. strcpy() can therefore write beyond the array and corrupt nearby memory. A larger character array, such as char phone[16], is safer.
- Incorrect blood-pressure representation: Blood pressure contains two readings, systolic and diastolic, but one float stores only one numeric value. A value such as 120/80 cannot be represented correctly as a single float. The two readings should be separate integer fields in a nested structure.
- Date stored as text: char visitDate[100][8] is too small for a safely null-terminated date string in common formats. It also makes date validation and date comparisons difficult. A date structure with day, month and year is more reliable.
- No capacity check: count is used as an array index without checking whether it is below 100. Adding more than 100 patients causes an out-of-bounds write and undefined behaviour.
- Global variables: The arrays and count are global, so any function can change them. This makes the program harder to test, maintain and protect from accidental modification.
- Sorting difficulty: Sorting by age requires moving corresponding elements in every parallel array. If even one array is not moved correctly, patient records become inconsistent. An array of structures allows the complete patient record to move together.
- Unvalidated input: The function does not check whether the name, phone number, age, date or blood-pressure values are valid. Incorrect information may enter the health-centre register.

1.2 Repaired Structure Design

Nested structures keep related values together. The patient structure contains a date structure and a blood-pressure structure. The function receives a pointer so it can update the selected patient record.

```
#include <stdio.h>
#include <string.h>
#define MAX_PATIENTS 100
typedef struct {
   int day, month, year;
} Date;
typedef struct {
   int systolic;
   int diastolic;
} BloodPressure;
typedef struct {
   char name[50];
   int age;
   char phone[16];
   BloodPressure bp;
   Date visitDate;
} Patient;
int addPatient(Patient patients[], int *count,
```

```
            const char *name, int age, const char *phone,
           BloodPressure bp, Date date)
{
   if (*count >= MAX_PATIENTS)
      return 0;
   Patient *p = &patients[*count];
   snprintf(p->name, sizeof(p->name), "%s", name);
   p->age = age;
   snprintf(p->phone, sizeof(p->phone), "%s", phone);
   p->bp = bp;
   p->visitDate = date;
   (*count)++;
   return 1;
}
int main()
{
   Patient patients[MAX_PATIENTS];
   int count = 0;
   BloodPressure bp = {120, 80};
   Date date = {12, 11, 2026};
   if (addPatient(patients, &count,
                "Arun", 25, "9876543210", bp, date))
   {
      printf("Patient added successfully!\n");
      printf("\nPatient Details\n");
      printf("Name: %s\n", patients[0].name);
      printf("Age: %d\n", patients[0].age);
      printf("Phone: %s\n", patients[0].phone);
      printf("Blood Pressure: %d/%d\n",
            patients[0].bp.systolic,
            patients[0].bp.diastolic);
      printf("Visit Date: %02d-%02d-%04d\n",
            patients[0].visitDate.day,
            patients[0].visitDate.month,
            patients[0].visitDate.year);
   }
   else
   {
      printf("Patient list is full!\n");
   }
   return 0;
}
```



1.3 Design Defence

An array of structures stores every patient's details as one unit. Sorting moves the complete record, preventing mismatched fields. A function can receive one structure or a pointer to it instead of receiving many separate arrays and parameters. Nested structures group reusable concepts such as dates and blood pressure. New fields can be added in one structure without creating and synchronising another parallel array. The design is therefore easier to maintain, extend, pass to functions and sort.

Alternative Concept and Complexity Analysis

Parallel arrays may be preferred when processing one field at a time, when a system-of-arrays layout improves cache use, or when different fields are stored or transferred separately. In an array of structures, a swap moves one complete Patient object; approximately sizeof(Patient) bytes are involved in each swap (or three structure-sized assignments in a conventional temporary-variable swap). In parallel arrays, a swap by age must exchange the age and every related field, moving the combined sizes of all fields. Both comparison-based sorting approaches generally take $O(n^2)$ time for a simple sort such as bubble sort or selection sort, or $O(n \log n)$ for an efficient comparison sort. Both use $O(n)$ storage for n records. Structure padding may add a few bytes to each Patient; parallel arrays can have less padding between individual fields, but require careful synchronisation.

TASK 2 – FROM REQUIREMENT TO NESTED STRUCTURES

2.1 Identify the Entities

The main entity is Parcel. It contains an ID, weight, booking date, sender, receiver, service type and charge. Sender and Receiver each contain an Address. Address contains city and PIN code. Date contains day, month and year.

2.2 Define the Structures

```
#include <stdio.h>
#include <string.h>

typedef struct {
   int day, month, year;
} Date;

typedef struct {
   char name[40];
   char city[30];
   int pincode;
} Person;

typedef enum { NORMAL, EXPRESS } ServiceType;

typedef struct {
   int id;
   float weight;
   Date booked;
   Person sender;
   Person receiver;
   ServiceType service;
   float charge;
} Parcel;

#define MAX_PARCELS 100
Parcel bookings[MAX_PARCELS];
```

2.3 Design the Operations

(a) Sort parcels by weight – selection-sort pseudocode:

```
FOR i = 0 TO n - 2
   minIndex = i
   FOR j = i + 1 TO n - 1
      IF parcels[j].weight < parcels[minIndex].weight
         minIndex = j
   END FOR
   IF minIndex != i
      swap the complete Parcel structures at i and minIndex
   END IF
END FOR
```

(b) List parcels for a given receiver city:

```
FOR i = 0 TO n - 1
   IF parcels[i].receiver.city matches the required city
      display the parcel ID, weight, receiver name and PIN
   END IF
```

END FOR

2.4 Access Paths

For the third parcel (array index 2):

(i) Array and dot operator:
list[2].to.pincode

(ii) Structure pointer and arrow operator:
(p + 2)->to.pincode
/* where p points to the first parcel */

Completed Code

```
#include <stdio.h>

struct date { int day, month, year; };
struct address { char city[20]; int pincode; };
struct parcel {
   int id;
   float weight;
   struct address to;
   struct date booked;
};

void print_parcel(const struct parcel *p)
{
   printf("%d %.1f kg %s-%d %02d/%02d/%04d\n",
          p->id, p->weight, p->to.city, p->to.pincode,
          p->booked.day, p->booked.month, p->booked.year);
}

void sort_by_weight(struct parcel p[], int n)
{
   for (int i = 0; i < n - 1; i++) {
      int min = i;
      for (int j = i + 1; j < n; j++)
         if (p[j].weight < p[min].weight) min = j;

      if (min != i) {
         struct parcel temp = p[i];
         p[i] = p[min];
         p[min] = temp;
      }
   }
}

int main(void)
{
   struct parcel list[3] = {
      {501, 4.5f, {"Madurai", 625001}, {12, 10, 2026}},
      {502, 1.2f, {"Chennai", 600077}, {12, 10, 2026}},
      {503, 2.8f, {"Coimbatore", 641001}, {13, 10, 2026}}
   };

   sort_by_weight(list, 3);
   for (int i = 0; i < 3; i++) print_parcel(&list[i]);
```

```
    return 0;
}
```

```c
#include <stdio.h>
struct date { int day, month, year; };
struct address { char city[20]; int pincode; };
struct parcel {
    int id;
    float weight;
    struct address to;
    struct date booked;
};
void print_parcel(const struct parcel *p)
{
    printf("%d %.1f kg %s-%d %02d/%02d/%04d\n",
           p->id, p->weight, p->to.city, p->to.pincode,
           p->booked.day, p->booked.month, p->booked.year);
}
void sort_by_weight(struct parcel p[], int n)
{
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (p[j].weight < p[min].weight) min = j;

        if (min != i) {
            struct parcel temp = p[i];
            p[i] = p[min];
            p[min] = temp;
        }
    }
}
int main(void)
{
    struct parcel list[3] = {
        {501, 4.5f, {"Madurai", 625001}, {12, 10, 2026}},
        {502, 1.2f, {"Chennai", 600077}, {12, 10, 2026}},
        {503, 2.8f, {"Coimbatore", 641001}, {13, 10, 2026}}
    };
    sort_by_weight(list, 3);
    for (int i = 0; i < 3; i++) print_parcel(&list[i]);
    return 0;
}
```

```
502 1.2 kg Chennai-600077 12/10/2026
503 2.8 kg Coimbatore-641001 13/10/2026
501 4.5 kg Madurai-625001 12/10/2026
```

Each swap uses a temporary Parcel and performs three structure assignments. Each assignment copies sizeof(struct parcel) bytes, so the total data copied in a swap is approximately 3 × sizeof(struct parcel).

Alternative Concept and Complexity Analysis

A flat structure would place receiver city and PIN, and booking day, month and year, directly inside Parcel. Its access paths would be list[2].pincode and p->pincode rather than list[2].to.pincode and p->to.pincode. A flat structure may be simpler for one specific parcel design, while nested Address and Date structures are reusable in other programs. The total data-member storage is the same; sizeof may differ because of alignment and padding. On a typical system where int and float are 4 bytes, the given small example's struct address is commonly 24 bytes, struct date 12 bytes and struct parcel 44 bytes, but actual sizes should be checked using sizeof on the target compiler.

TASK 3 – UNIONS AND MEMORY LAYOUT

3.1 Structure or Union?

A structure containing float temperature, int humidity and char status[8] reserves space for all three members at the same time. A union shares the same memory among its members, so it stores one value at a time. Since each weather packet contains exactly one kind of value and memory is limited, a tagged union is the suitable design. The tag identifies which union member currently contains valid data.

Structure and Tagged Union Definitions

```
struct all_values {
   float celsius;
   int percent;
   char status[8];
};

enum kind { TEMPERATURE, HUMIDITY, STATUS };

struct reading {
   enum kind type;
   union {
      float celsius;
      int percent;
      char status[8];
   } value;
};
```

3.2 Predict the Size

On the typical 64-bit GCC system specified in the assignment, the union size is 8 bytes because its largest member is char status[8]. The union's alignment is normally 4 bytes. The enum tag is commonly 4 bytes. Therefore, the structure has 4 bytes for the tag followed by 8 bytes for the union, giving 12 bytes. The structure does not reserve separate space for all union members. Actual sizes depend on the compiler and ABI, so sizeof is the final verification.

```
printf("sizeof union = %zu, sizeof struct reading = %zu\\n",
      sizeof log[0].value, sizeof log[0]);
```

Expected typical output: sizeof union = 8, sizeof struct reading = 12.

3.3 Implement and Test

```
#include <stdio.h>

enum kind {
   TEMPERATURE,
   HUMIDITY,
   STATUS
};

struct reading {
   enum kind type;

   union {
```

```
        float celsius;
        int percent;
        char status[8];
    } value;
};

void show(const struct reading *r)
{
    switch (r->type) {

        case TEMPERATURE:
            printf("Temp %.1f C\n", r->value.celsius);
            break;

        case HUMIDITY:
            printf("Humidity %d %%\n", r->value.percent);
            break;

        case STATUS:
            printf("Status %s\n", r->value.status);
            break;

        default:
            printf("Invalid reading\n");
    }
}

int main(void)
{
    struct reading log[3] = {
        {TEMPERATURE, {.celsius = 31.5f}},
        {HUMIDITY, {.percent = 68}},
        {STATUS, {.status = "OK"}}
    };

    printf("sizeof union = %zu, sizeof struct reading = %zu\n",
           sizeof log[0].value,
           sizeof log[0]);

    printf("\nSensor Readings:\n");

    for (int i = 0; i < 3; i++) {
        show(&log[i]);
    }

    return 0;
}
```

```c
#include <stdio.h>
enum kind {
    TEMPERATURE,
    HUMIDITY,
    STATUS
};
struct reading {
    enum kind type;

    union {
        float celsius;
        int percent;
        char status[8];
    } value;
};
void show(const struct reading *r)
{
    switch (r->type) {
        case TEMPERATURE:
            printf("Temp %.1f C\n", r->value.celsius);
            break;
        case HUMIDITY:
            printf("Humidity %d %%\n", r->value.percent);
            break;
        case STATUS:
            printf("Status %s\n", r->value.status);
            break;
        default:
            printf("Invalid reading\n");
    }
}

int main(void)
{
    struct reading log[3] = {
        {TEMPERATURE, {.celsius = 31.5f}},
        {HUMIDITY, {.percent = 68}},
        {STATUS, {.status = "OK"}}
    };
    printf("sizeof union = %zu, sizeof struct reading = %zu\n",
           sizeof log[0].value,
           sizeof log[0]);
    printf("\nSensor Readings:\n");
    for (int i = 0; i < 3; i++) {
        show(&log[i]);
    }
    return 0;
```

```
sizeof union = 8, sizeof struct reading = 12

Sensor Readings:
Temp 31.5 C
Humidity 68 %
Status OK
```

3.4 Misuse Experiment

If a packet is written using the temperature member and then read using the humidity member, the program interprets the same stored bytes as an int. The result is not a meaningful humidity reading and is not portable. The tag must always be checked before reading a union member. The tag and active member must be updated together.

Why the Structure Is 12 Bytes

The tag occupies 4 bytes. The union occupies 8 bytes because its largest member is the 8-byte status array. The union begins at an address aligned for its members. Thus the total is 4 + 8 = 12 bytes on the stated typical system, rather than adding the sizes of every union member.

Alternative Concept and Complexity Analysis

With three separate arrays, each packet type would be stored in its own array. Reading a known packet can be O(1), but the program needs a type-to-array mapping and a way to preserve the overall packet order. A void-pointer design can allocate each value separately, but needs dynamic allocation, explicit type information, pointer management and freeing. Reading is O(1) after locating the pointer, but allocation and pointer overhead increase memory use. A tagged union stores the tag and the largest member's storage inline, avoiding per-packet allocation and keeping packets together.

TASK 4 – STRUCTURE FORENSICS

4.1 Locate the Defects

The addInterest function receives struct account a by value. C creates a copy of the structure, so changing a.balance changes only the local copy. The original element in list remains unchanged. The statement p.number and p.balance is invalid because p is a pointer; the arrow operator must be used as p->number and p->balance.

4.2 Construct Evidence

```
#include <stdio.h>

struct account { int number; float balance; };

void addInterest(struct account a, float rate)
{
   printf("Address inside function: %p\n", (void *)&a);
   a.balance += a.balance * rate / 100.0f;
}

int main(void)
{
   struct account list[3] = {
      {101, 5000}, {102, 12000}, {103, 800}
   };
   printf("Address in main: %p\n", (void *)&list[0]);
   addInterest(list[0], 4);
   return 0;
}
```



The address printed for the local parameter copy is different from the address of list[0]. Exact address values vary on each run.

4.3 Repaired Program

```
#include <stdio.h>

struct account { int number; float balance; };

void addInterest(struct account *a, float rate)
{
   a->balance += a->balance * rate / 100.0f;
}

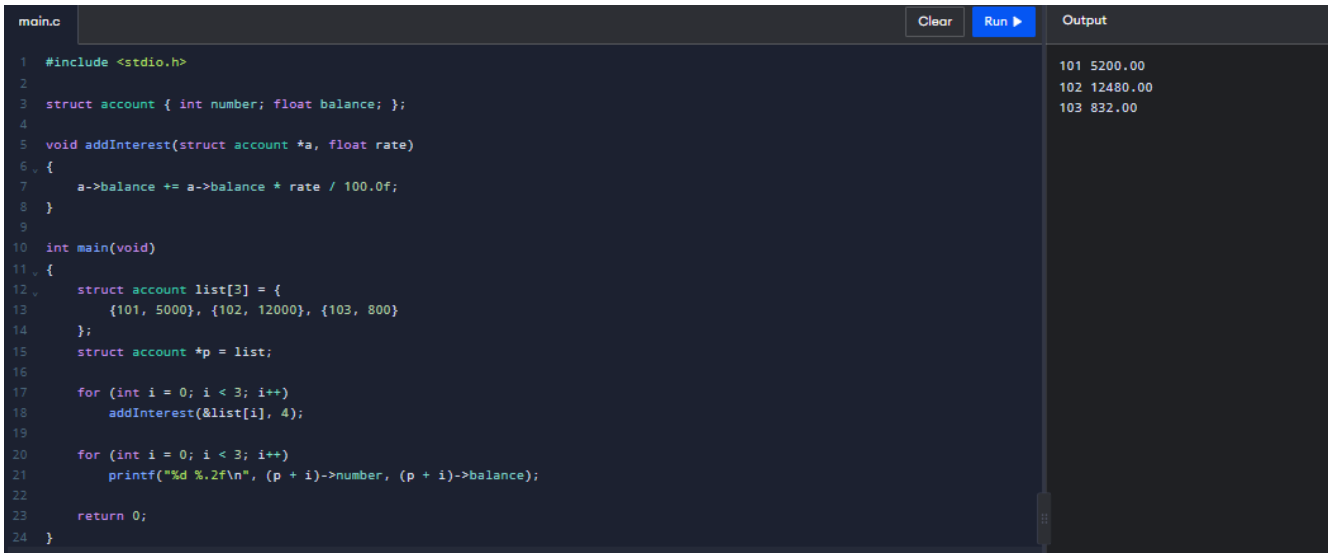
int main(void)
```

```
{
    struct account list[3] = {
        {101, 5000}, {102, 12000}, {103, 800}
    };
    struct account *p = list;

    for (int i = 0; i < 3; i++)
        addInterest(&list[i], 4);

    for (int i = 0; i < 3; i++)
        printf("%d %.2f\n", (p + i)->number, (p + i)->balance);

    return 0;
}
```



4.4 Prevent Recurrence

- Pass by value when the function only needs a small independent copy and must not modify the caller's original structure.
- Pass by pointer when the function must modify the original structure or when copying a large structure should be avoided.
- Pass by const pointer when the function only reads the structure. The const qualifier prevents modification through that pointer and avoids copying the complete structure.

Alternative Concept and Complexity Analysis

The alternative function can accept a structure by value, modify its local copy and return the result. The caller then assigns the returned structure to list[i]. This approach copies sizeof(struct account) bytes into the function and returns a structure-sized value; the compiler may optimise some copies. Passing a pointer passes only an address (commonly 8 bytes on a 64-bit system) and directly updates the original. Both take O(1) time for a fixed-size account, but by-value passing uses temporary storage for the copy while pointer passing uses only pointer-sized parameter storage.

TASK 5 – STORAGE CLASSES: THE SAME COUNTER FOUR WAYS

5.1 Four Implementations

(a) Automatic local variable: it is created again on every call, so each call starts from 1000 and returns 1001.

```
int auto_token(void)
{
   int token = 1000;
   return ++token;
}
```

(b) Static local variable: it is initialised once and retains its value between calls.

```
int next_token(void)
{
   static int token = 1000;
   return ++token;
}
```

(c) Global variable shared across two source files:

```
/* counter.c */
int token_counter = 1000;

int next_global_token(void)
{
   return ++token_counter;
}

/* main.c */
extern int token_counter;
int next_global_token(void);
```

The extern declaration tells main.c that token_counter is defined in another source file. Compile and link both files together.

(d) Register loop variable:

```
for (register int i = 0; i < 3; i++) {
   printf("%d\n", i);
}
```

The register keyword requests that the compiler consider keeping the frequently used loop variable in a CPU register. It is only a hint; modern compilers decide the actual storage.

```
#include <stdio.h>

// Global variable

int token_counter = 1000;

// (a) Automatic local variable

int auto_token(void)

{

   int token = 1000;

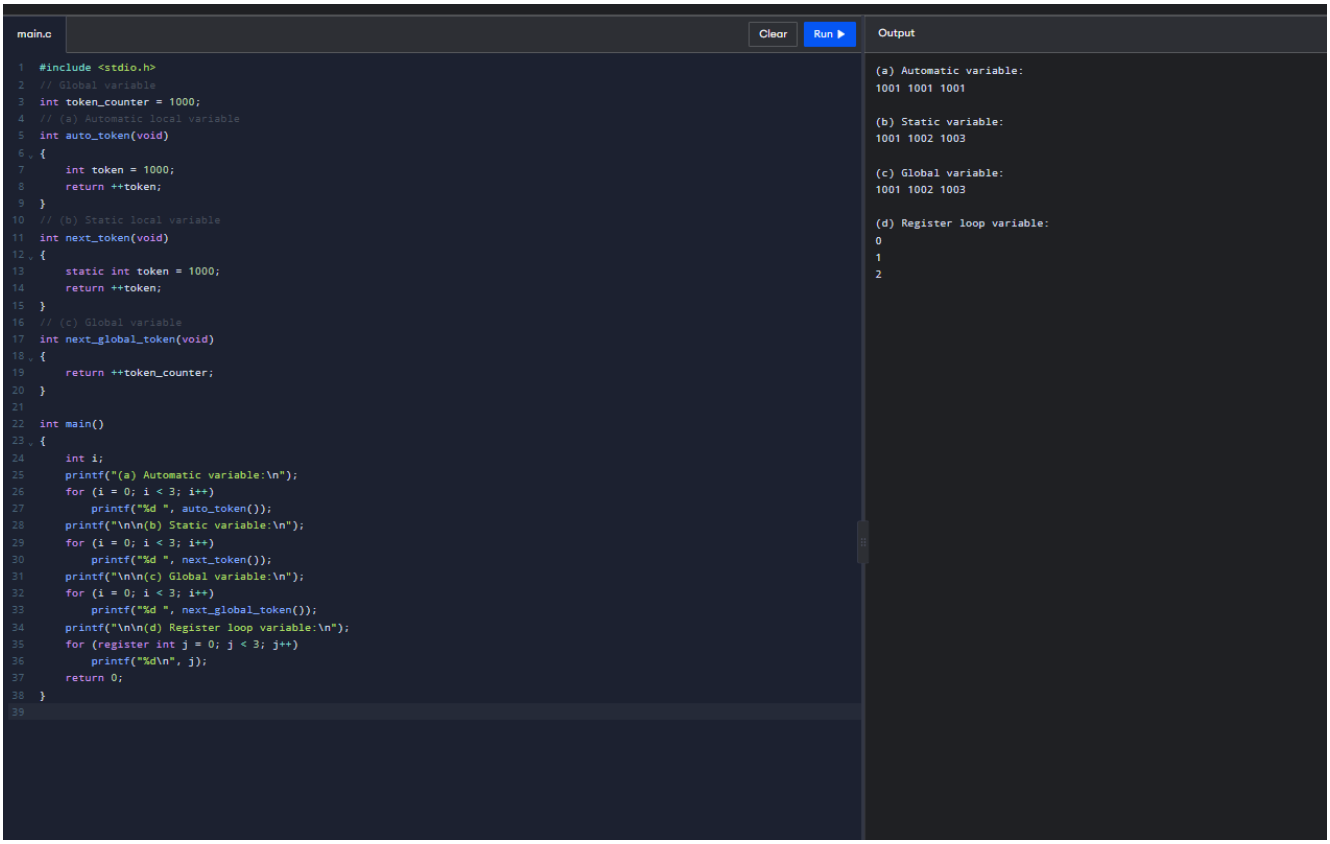
   return ++token;

}

// (b) Static local variable
```

```
int next_token(void)
{
   static int token = 1000;
   return ++token;
}
// (c) Global variable
int next_global_token(void)
{
   return ++token_counter;
}

int main()
{
   int i;
   printf("(a) Automatic variable:\n");
   for (i = 0; i < 3; i++)
      printf("%d ", auto_token());
   printf("\n\n(b) Static variable:\n");
   for (i = 0; i < 3; i++)
      printf("%d ", next_token());
   printf("\n\n(c) Global variable:\n");
   for (i = 0; i < 3; i++)
      printf("%d ", next_global_token());
   printf("\n\n(d) Register loop variable:\n");
   for (register int j = 0; j < 3; j++)
      printf("%d\n", j);
   return 0;
}
```



5.2 Change the Requirement – Two Independent Counters

With static local variables, create two separate functions, such as next_veg_token() and next_nonveg_token(), each with its own static counter. With globals, create two variables, veg_counter and nonveg_counter. With caller-owned counters, pass the address of the required counter to the function. The automatic version still resets on each call and is unsuitable unless the caller stores and updates the value. The register loop variable is only a loop-control variable and should not be used as persistent shared state.

```
int next_token(int *counter)
{
   ++(*counter);
   return *counter;
}
/* Example */
int veg = 1000, nonveg = 2000;
int v = next_token(&veg);
int nv = next_token(&nonveg);
```

5.3 Compare the Storage Classes

Automatic: block scope, normally exists only while the block/function is executing, and has an indeterminate initial value if not initialised. A local automatic variable is recreated on each function call.

Static local: block scope, exists for the entire program execution, and is initialised to zero by default. Its value is retained between calls.

Global (file-scope) variable: file scope by default, exists for the entire program execution, and is zero-initialised by default. Another source file can refer to it using extern when it has external linkage.

Register: block scope and automatic lifetime, with an initial value that is indeterminate if not initialised. register is a storage hint. The address-of operator (&) cannot be applied to a register variable in standard C because the programmer is not allowed to request its memory address.

Risk of accidental modification: globals are most exposed because many functions and files may access them. Static locals are hidden within their function but retain state. Automatic variables have limited scope and lifetime. Register variables also have limited scope, but the keyword does not guarantee a physical CPU register.

Function Argument Evaluation Order

In C, the order in which function arguments are evaluated is unspecified. Therefore, placing three calls that modify the same counter inside one printf can produce an order that should not be relied upon. Call next_token() in separate statements and store each result before printing.

Alternative Concept and Complexity Analysis

A caller-owned counter is passed by pointer, as shown above. Incrementing it takes O(1) time and the function uses O(1) extra space. Its lifetime is controlled by the caller, and it can support any number of independent counters simply by passing different addresses. This avoids hidden global or static state and makes testing easier.

TASK 6 – DESIGN CHALLENGE: COLLEGE CANTEEN INVENTORY AND BILLING

Stage 1 – Interpret the Problem

Entities: Item, PackagedItem details, CookedItem details, ComboItem details, Date, SaleRecord and BillSummary. Every item has an ID, name, price, stock quantity and a kind. Packaged items have an expiry date. Cooked items have a preparation time. Combo meals contain up to three component item IDs and quantities.

Testable requirements:

- An item can be added with a unique ID and valid price and stock quantity.
- A sale is accepted only when sufficient stock is available.
- After a successful sale, stock decreases by the quantity sold and revenue increases by price multiplied by quantity.

- An attempted sale greater than available stock is rejected without changing stock or revenue.
- Packaged items whose expiry date is today or within the next two days are listed as warnings.
- Combo meals contain no more than three component items and stock is checked for each component.
- The daily bill summary is sorted by revenue in descending order.

Stage 2 – Proposed Data Model

Use a Date structure for expiry dates and a tagged union for the three item kinds. An array of Item structures stores the inventory. A SaleRecord array stores sales for the day. The tag determines whether the union's packaged, cooked or combo member is valid.

Stage 3 – Critical Design Decisions

- Stock must be checked before any quantity is deducted. For a combo, check every component first, then deduct all components, so a partial sale cannot occur.
- Expiry dates should be compared as year, month and day values using a date structure, not as unvalidated text.
- The inventory and sales should normally be passed to functions with pointers and an explicit count. Global variables are not necessary. A static variable may be used only if a persistent unique sale number is specifically required.

Stage 4 – Structure and Function Design

```
#include <stdio.h>
#include <string.h>

#define MAX_ITEMS 100
#define MAX_SALES 500

typedef struct {
   int day, month, year;
} Date;

typedef struct {
   Date expiry;
} PackagedInfo;

typedef struct {
   int hour, minute;
} CookedInfo;

typedef struct {
   int component_id[3];
   int component_qty[3];
   int count;
} ComboInfo;

typedef enum { PACKAGED, COOKED, COMBO } ItemKind;

typedef struct {
   int id;
   char name[40];
   float price;
   int stock;
```

```
    ItemKind kind;
    union {
        PackagedInfo packaged;
        CookedInfo cooked;
        ComboInfo combo;
    } detail;
} Item;

typedef struct {
    int item_id;
    int quantity;
    float revenue;
} SaleRecord;

int find_item(const Item items[], int n, int id);
int record_sale(Item items[], int n, SaleRecord sales[],
                int *sale_count, int id, int quantity);
int is_within_two_days(Date expiry, Date today);
void show_expiry_warnings(const Item items[], int n, Date today);
void sort_sales_by_revenue(SaleRecord sales[], int n);
```

Stage 5 – Implementation Logic and Demonstration

The following pseudocode describes safe sale processing. For combo sales, first validate all component IDs and quantities and confirm stock for every component. Only after all checks pass should stock be reduced.

```
RECORD SALE(item ID, quantity)
    Find the requested item.
    If item does not exist or quantity <= 0, reject.
    If item is a packaged or cooked item:
        If stock < quantity, reject.
        Otherwise reduce stock by quantity.
    If item is a combo:
        Check that it has at most three components.
        Check stock for every component multiplied by quantity.
        If any component has insufficient stock, reject without changes.
        Otherwise reduce stock for every component.
    Calculate revenue and append a SaleRecord.
    Confirm the sale.
END

EXPIRY WARNING(today)
    For each inventory item:
        If item is PACKAGED and expiry is today through today + 2 days:
            Display item ID, name and expiry date.
END

SORT DAILY SUMMARY
    Sort SaleRecord entries by revenue in descending order.
    Print each item ID, quantity and revenue.
END
#include <stdio.h>
#include <string.h>

#define MAX_ITEMS 100
#define MAX_SALES 500

/* DATE STRUCTURE */
```

```
typedef struct {
   int day, month, year;
} Date;

/* ITEM DETAILS */
typedef struct {
   Date expiry;
} PackagedInfo;

typedef struct {
   int hour, minute;
} CookedInfo;

typedef struct {
   int component_id[3];
   int component_qty[3];
   int count;
} ComboInfo;

/* ENUMERATION */
typedef enum {
   PACKAGED,
   COOKED,
   COMBO
} ItemKind;

/* ITEM STRUCTURE WITH UNION */
typedef struct {
   int id;
   char name[40];
   float price;
   int stock;
   ItemKind kind;

   union {
      PackagedInfo packaged;
      CookedInfo cooked;
      ComboInfo combo;
   } detail;

} Item;

/* SALES STRUCTURE */
typedef struct {
   int item_id;
   int quantity;
   float revenue;
} SaleRecord;


/* FUNCTION TO CHECK LEAP YEAR */
int is_leap_year(int year)
{
   return (year % 400 == 0 ||
         (year % 4 == 0 && year % 100 != 0));
}

/* FUNCTION TO VALIDATE DATE */
```

```
int valid_date(Date d)
{
   int days[] = {
      31, 28, 31, 30, 31, 30,
      31, 31, 30, 31, 30, 31
   };

   if (d.year < 1 || d.month < 1 || d.month > 12)
      return 0;

   if (is_leap_year(d.year))
      days[1] = 29;

   return d.day >= 1 && d.day <= days[d.month - 1];
}

/* CONVERT DATE INTO DAY COUNT */
long date_to_days(Date d)
{
   long total = 0;

   for (int y = 1; y < d.year; y++) {
      total += is_leap_year(y) ? 366 : 365;
   }

   int days[] = {
      31, 28, 31, 30, 31, 30,
      31, 31, 30, 31, 30, 31
   };

   if (is_leap_year(d.year))
      days[1] = 29;

   for (int m = 0; m < d.month - 1; m++) {
      total += days[m];
   }

   total += d.day;

   return total;
}

/* FIND ITEM USING ID */
int find_item(const Item items[], int n, int id)
{
   for (int i = 0; i < n; i++) {
      if (items[i].id == id)
         return i;
   }

   return -1;
}

/* ADD ITEM */
int add_item(Item items[], int *n, Item new_item)
{
   if (*n >= MAX_ITEMS) {
      printf("Inventory is full!\n");
```

```
        return 0;
    }

    if (find_item(items, *n, new_item.id) != -1) {
        printf("Item ID already exists!\n");
        return 0;
    }

    if (new_item.id <= 0 ||
        new_item.price <= 0 ||
        new_item.stock < 0) {
        printf("Invalid item details!\n");
        return 0;
    }

    items[*n] = new_item;
    (*n)++;

    printf("Item added successfully!\n");
    return 1;
}

/* CHECK STOCK AND RECORD SALE */
int record_sale(Item items[], int n,
                SaleRecord sales[],
                int *sale_count,
                int id, int quantity)
{
    int index = find_item(items, n, id);

    if (index == -1 || quantity <= 0) {
        printf("Invalid item ID or quantity!\n");
        return 0;
    }

    if (*sale_count >= MAX_SALES) {
        printf("Sales record is full!\n");
        return 0;
    }

    Item *item = &items[index];

    /* NORMAL ITEM SALE */
    if (item->kind == PACKAGED || item->kind == COOKED) {

        if (item->stock < quantity) {
            printf("Sale rejected! Insufficient stock.\n");
            return 0;
        }

        item->stock -= quantity;
    }

    /* COMBO ITEM SALE */
    else if (item->kind == COMBO) {

        ComboInfo *c = &item->detail.combo;
```

```
        if (c->count < 1 || c->count > 3) {
            printf("Invalid combo configuration!\n");
            return 0;
        }

        /* CHECK ALL COMPONENTS FIRST */
        for (int i = 0; i < c->count; i++) {

            int component_index =
                find_item(items, n, c->component_id[i]);

            if (component_index == -1 ||
                items[component_index].kind == COMBO ||
                c->component_qty[i] <= 0) {

                printf("Invalid combo component!\n");
                return 0;
            }

            if (c->component_qty[i] > 0 &&
                quantity > items[component_index].stock /
                        c->component_qty[i]) {

                printf("Sale rejected! Insufficient component stock.\n");
                return 0;
            }
        }

        /* DEDUCT STOCK ONLY AFTER ALL CHECKS PASS */
        for (int i = 0; i < c->count; i++) {

            int component_index =
                find_item(items, n, c->component_id[i]);

            items[component_index].stock -=
                c->component_qty[i] * quantity;
        }
    }

    else {
        printf("Invalid item kind!\n");
        return 0;
    }

    /* CALCULATE REVENUE */
    sales[*sale_count].item_id = id;
    sales[*sale_count].quantity = quantity;
    sales[*sale_count].revenue = item->price * quantity;

    (*sale_count)++;

    printf("Sale successful!\n");
    printf("Revenue: Rs. %.2f\n",
           item->price * quantity);

    return 1;
}
```

```
/* CHECK EXPIRY WITHIN TWO DAYS */
int is_within_two_days(Date expiry, Date today)
{
   if (!valid_date(expiry) || !valid_date(today))
      return 0;

   long difference =
      date_to_days(expiry) - date_to_days(today);

   return difference >= 0 && difference <= 2;
}

/* DISPLAY EXPIRY WARNINGS */
void show_expiry_warnings(const Item items[],
                          int n, Date today)
{
   int found = 0;

   printf("\n--- EXPIRY WARNINGS ---\n");

   for (int i = 0; i < n; i++) {

      if (items[i].kind == PACKAGED &&
         is_within_two_days(
            items[i].detail.packaged.expiry, today)) {

         Date d = items[i].detail.packaged.expiry;

         printf("ID: %d | Name: %s | Expiry: %02d-%02d-%04d\n",
               items[i].id, items[i].name,
               d.day, d.month, d.year);

         found = 1;
      }
   }

   if (!found)
      printf("No items are expiring within two days.\n");
}

/* SORT SALES BY REVENUE */
void sort_sales_by_revenue(SaleRecord sales[], int n)
{
   for (int i = 0; i < n - 1; i++) {

      for (int j = 0; j < n - i - 1; j++) {

         if (sales[j].revenue < sales[j + 1].revenue) {

            SaleRecord temp = sales[j];
            sales[j] = sales[j + 1];
            sales[j + 1] = temp;
         }
      }
   }
}

/* DISPLAY DAILY BILL SUMMARY */
```

```
void show_bill_summary(const SaleRecord sales[],
                       int n, const Item items[], int item_count)
{
    SaleRecord sorted[MAX_SALES];

    if (n == 0) {
        printf("\nNo sales recorded today.\n");
        return;
    }

    for (int i = 0; i < n; i++)
        sorted[i] = sales[i];

    sort_sales_by_revenue(sorted, n);

    float total = 0;

    printf("\n========== DAILY BILL SUMMARY ==========\n");
    printf("%-10s %-20s %-10s %-12s\n",
           "Item ID", "Item Name", "Quantity", "Revenue");

    for (int i = 0; i < n; i++) {

        int index = find_item(items, item_count,
                              sorted[i].item_id);

        printf("%-10d %-20s %-10d Rs. %.2f\n",
               sorted[i].item_id,
               items[index].name,
               sorted[i].quantity,
               sorted[i].revenue);

        total += sorted[i].revenue;
    }

    printf("-----------------------------------------\n");
    printf("Total Daily Revenue: Rs. %.2f\n", total);
}

/* DISPLAY INVENTORY */
void display_items(const Item items[], int n)
{
    printf("\n========== INVENTORY ==========\n");

    for (int i = 0; i < n; i++) {

        printf("\nID: %d\n", items[i].id);
        printf("Name: %s\n", items[i].name);
        printf("Price: Rs. %.2f\n", items[i].price);

        if (items[i].kind == COMBO) {
            printf("Type: Combo\n");

            for (int j = 0;
                 j < items[i].detail.combo.count; j++) {

                printf("Component ID: %d, Quantity: %d\n",
                       items[i].detail.combo.component_id[j],
```

```
                items[i].detail.combo.component_qty[j]);
            }
        }
        else {
            printf("Stock: %d\n", items[i].stock);

            if (items[i].kind == PACKAGED) {
                Date d = items[i].detail.packaged.expiry;

                printf("Type: Packaged\n");
                printf("Expiry: %02d-%02d-%04d\n",
                    d.day, d.month, d.year);
            }
            else {
                printf("Type: Cooked\n");
                printf("Preparation Time: %02d:%02d\n",
                    items[i].detail.cooked.hour,
                    items[i].detail.cooked.minute);
            }
        }
    }
}

/* MAIN FUNCTION */
int main(void)
{
    Item items[MAX_ITEMS];
    SaleRecord sales[MAX_SALES];

    int n = 0;
    int sale_count = 0;
    int choice;

    Date today;

    printf("===== INVENTORY AND SALES MANAGEMENT SYSTEM =====\n");

    printf("Enter today's date (DD MM YYYY): ");
    if (scanf("%d %d %d",
            &today.day, &today.month, &today.year) != 3 ||
        !valid_date(today)) {
        printf("Invalid date!\n");
        return 1;
    }

    do {

        printf("\n========== MAIN MENU ==========\n");
        printf("1. Add Packaged Item\n");
        printf("2. Add Cooked Item\n");
        printf("3. Add Combo Meal\n");
        printf("4. Display Inventory\n");
        printf("5. Record Sale\n");
        printf("6. Expiry Warnings\n");
        printf("7. Daily Bill Summary\n");
        printf("0. Exit\n");

        printf("Enter your choice: ");
```

```
if (scanf("%d", &choice) != 1) {
   printf("Invalid input!\n");
   return 1;
}

Item new_item;

switch (choice) {

   case 1:
      new_item.kind = PACKAGED;

      printf("Enter ID, name, price and stock: ");

      if (scanf("%d %39s %f %d",
            &new_item.id, new_item.name,
            &new_item.price, &new_item.stock) != 4) {
         printf("Invalid input!\n");
         return 1;
      }

      printf("Enter expiry date (DD MM YYYY): ");

      if (scanf("%d %d %d",
            &new_item.detail.packaged.expiry.day,
            &new_item.detail.packaged.expiry.month,
            &new_item.detail.packaged.expiry.year) != 3 ||
         !valid_date(new_item.detail.packaged.expiry)) {
         printf("Invalid expiry date!\n");
         break;
      }

      add_item(items, &n, new_item);
      break;

   case 2:
      new_item.kind = COOKED;

      printf("Enter ID, name, price and stock: ");

      if (scanf("%d %39s %f %d",
            &new_item.id, new_item.name,
            &new_item.price, &new_item.stock) != 4) {
         printf("Invalid input!\n");
         return 1;
      }

      printf("Enter preparation time (hours minutes): ");

      if (scanf("%d %d",
            &new_item.detail.cooked.hour,
            &new_item.detail.cooked.minute) != 2 ||
         new_item.detail.cooked.hour < 0 ||
         new_item.detail.cooked.minute < 0 ||
         new_item.detail.cooked.minute > 59) {
         printf("Invalid preparation time!\n");
         break;
```

```
    }

    add_item(items, &n, new_item);
    break;

case 3:
    new_item.kind = COMBO;
    new_item.stock = 0;

    printf("Enter combo ID, name and price: ");

    if (scanf("%d %39s %f",
            &new_item.id, new_item.name,
            &new_item.price) != 3) {
        printf("Invalid input!\n");
        return 1;
    }

    printf("Enter number of components (1-3): ");

    if (scanf("%d",
            &new_item.detail.combo.count) != 1 ||
        new_item.detail.combo.count < 1 ||
        new_item.detail.combo.count > 3) {
        printf("Invalid component count!\n");
        break;
    }

    for (int i = 0;
         i < new_item.detail.combo.count; i++) {

        printf("Enter component ID and quantity: ");

        if (scanf("%d %d",
                &new_item.detail.combo.component_id[i],
                &new_item.detail.combo.component_qty[i]) != 2) {
            printf("Invalid input!\n");
            return 1;
        }

        if (new_item.detail.combo.component_qty[i] <= 0 ||
            find_item(items, n,
                    new_item.detail.combo.component_id[i]) == -1 ||
            items[find_item(items, n,
                    new_item.detail.combo.component_id[i])].kind == COMBO) {
            printf("Invalid component!\n");
            break;
        }

        int duplicate = 0;

        for (int j = 0; j < i; j++) {
            if (new_item.detail.combo.component_id[j] ==
                new_item.detail.combo.component_id[i]) {
                duplicate = 1;
            }
        }
```

```
            if (duplicate) {
                printf("Duplicate component ID!\n");
                break;
            }
        }

        add_item(items, &n, new_item);
        break;

    case 4:
        display_items(items, n);
        break;

    case 5: {
        int id, quantity;

        printf("Enter item ID: ");
        scanf("%d", &id);

        printf("Enter quantity: ");
        scanf("%d", &quantity);

        record_sale(items, n, sales,
                    &sale_count, id, quantity);
        break;
    }

    case 6:
        show_expiry_warnings(items, n, today);
        break;

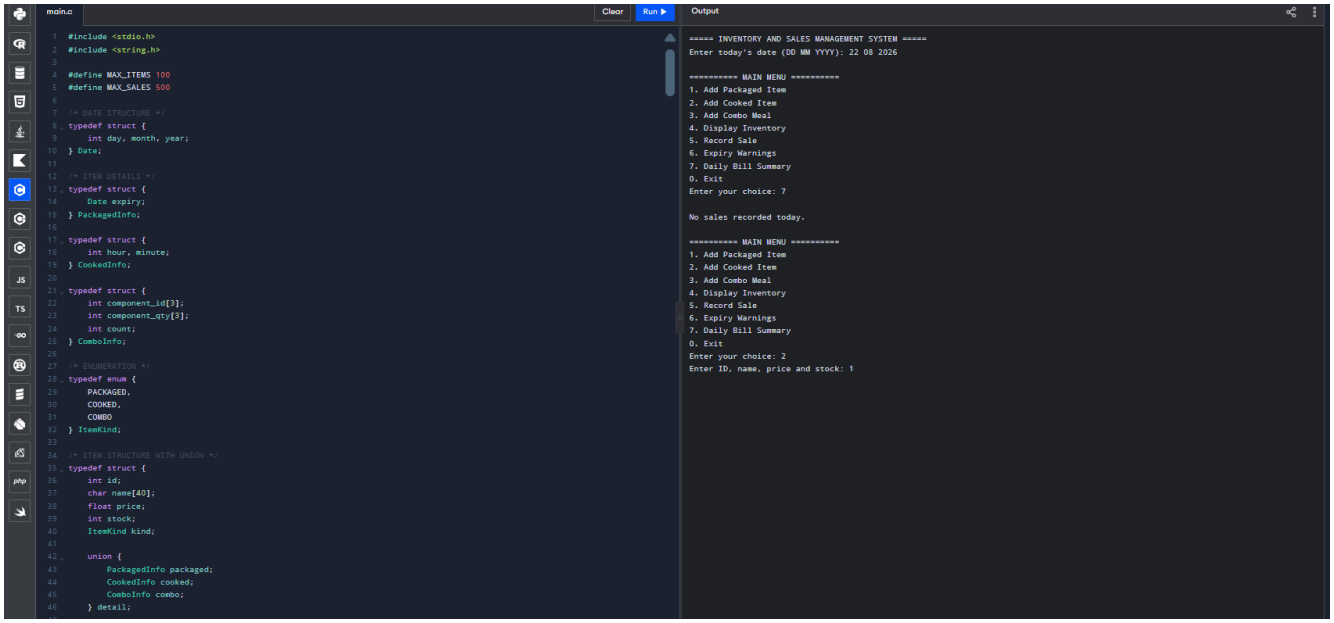
    case 7:
        show_bill_summary(sales, sale_count,
                          items, n);
        break;

    case 0:
        printf("Thank you! Program terminated.\n");
        break;

    default:
        printf("Invalid choice! Try again.\n");
    }

  } while (choice != 0);

  return 0;
}
```



Example test cases:

- Normal sale: an item costs ₹20 and has stock 10. Selling 2 units is accepted, stock becomes 8 and revenue is ₹40.
- Sale exceeding stock: if only 1 unit remains, a request to sell 2 is rejected. Stock and revenue remain unchanged.
- Expiring item: a packaged item expiring today, tomorrow or the day after tomorrow appears in the warning list.
- Sorted summary: sales with revenues ₹100, ₹40 and ₹75 are displayed in the order ₹100, ₹75 and ₹40.

Date comparison must account for month lengths and leap years. A production implementation should convert valid dates to a comparable day count or use a reliable date routine; simply comparing day numbers is not sufficient.

Alternative Concept and Complexity Analysis – Linked List

With a linked list, each item node stores an Item structure and a pointer to the next node. Inserting at the head takes O(1) time. Removing a node takes O(n) when the previous node must first be found, while searching by item ID takes O(n). An array also needs O(n) search, but insertion or removal in the middle may require shifting elements. Arrays provide contiguous storage and good cache performance; linked lists allow flexible growth without shifting. Each linked-list node uses extra memory for its next pointer and may also incur allocator overhead and memory fragmentation. The linked list is useful when frequent insertions and removals are required; an array is simpler when the maximum inventory size is known.